UNIVERSIDADE SANTO AMARO – UNISA

PROJETO INTEGRADOR: LABORATÓRIO DE PROGRAMAÇÃO

CONTROLE DE MATERIAIS DO LABORATÓRIO

Programa introdutório para registrar materiais e saídas de uso

Estudante(s):

_____

Curso: _____ Turma/semestre: _____

Professor(a) orientador(a): _____

Repositório do código-fonte:

https://github.com/franciellyneyane/controle-materiais-laboratorio

São Paulo

2026

SUMÁRIO

Seção	Pág.
Apresentação do projeto	3
1 Introdução e problema	4
2 Objetivos e critérios de sucesso	5
3 Fundamentação: controle de materiais	6
4 Fundamentação: conceitos iniciais de Python	7
5 Metodologia e cenário	8
6 Requisitos e delimitação do escopo	9
7 Modelo de dados e lógica do programa	10
8 Organização e implementação do código	11
9 Guia de execução e operação	12
10 Testes e resultados	13
11 Limitações, segurança e melhorias	14
12 Considerações finais	15
Referências	16
Apêndice A — Código-fonte	17

APRESENTAÇÃO DO PROJETO

Este documento apresenta o Controle de Materiais do Laboratório, um programa de terminal elaborado como exercício introdutório de programação. A aplicação permite registrar itens, consultar quantidades, lançar entradas e saídas de uso e destacar materiais cuja quantidade atingiu o limite mínimo definido pelo usuário. O projeto foi mantido deliberadamente pequeno para que a lógica possa ser acompanhada por uma pessoa que está começando a programar.

A situação representada é didática e simulada. Não houve coleta de dados em um laboratório real, entrevista com funcionários nem implantação em uma instituição. O cenário foi escolhido para tornar concreto um problema simples: saber quais materiais estão cadastrados e perceber quando o saldo informado chegou a um limite de atenção. O manual da disciplina admite ambientes simulados quando alinhados com o professor orientador; essa aprovação deve ser confirmada antes de apresentar o trabalho como solução de uma organização específica [5].

O programa foi desenvolvido em Python 3 e utiliza recursos básicos da linguagem: variáveis, listas, dicionários, funções, decisões condicionais, laços de repetição, entrada e saída de texto e tratamento de valores inválidos. Os dados permanecem somente na memória enquanto o programa está aberto. Ao encerrar a execução, a lista é perdida; por isso, o resultado deve ser entendido como protótipo para aprendizagem e demonstração, não como sistema pronto para operação profissional.

A documentação reúne a justificativa do tema, a fundamentação bibliográfica, os objetivos, o escopo, os requisitos, a lógica de funcionamento, a explicação das funções, um roteiro de uso e os resultados de testes simples. O código-fonte pode ser consultado no repositório indicado na capa e é reproduzido no Apêndice A para facilitar a leitura quando o relatório for enviado como um único arquivo.

1 INTRODUÇÃO E PROBLEMA

1.1 Contexto

Materiais de uso compartilhado podem incluir cabos, adaptadores, periféricos e outros itens de apoio. Mesmo em um cenário pequeno, uma informação desatualizada pode dificultar a localização de um item ou a percepção de que ele está acabando. Um registro simples de quantidade não substitui um processo administrativo completo, mas pode ajudar a organizar as informações básicas que alguém consulta durante uma rotina de empréstimo ou reposição.

1.2 Problema do projeto

A questão orientadora é: como um programa introdutório pode apoiar o registro de materiais de um laboratório e indicar quando a quantidade disponível chega ao limite mínimo definido? A pergunta delimita um problema que pode ser representado com entradas numéricas e regras claras: cadastrar o nome do item, guardar uma quantidade inicial, registrar acréscimos ou retiradas e comparar o saldo com o mínimo informado.

1.3 Justificativa

O tema foi escolhido porque permite relacionar uma situação cotidiana a estruturas iniciais de programação. Em vez de começar com um sistema extenso, o estudante pode observar como uma lista reúne vários materiais e como cada dicionário guarda os dados de um item. A gestão de estoque é relevante também para pequenos negócios: o Sebrae apresenta o tema como apoio para reduzir perdas e custos, organizar compras e manter disponibilidade, além de tratar de métodos mais amplos de controle [1]. O protótipo não implementa tais métodos; usa apenas um saldo e um limite simples.

A proposta também tem valor didático. O problema exige decisões compreensíveis: como impedir uma saída maior que o saldo, como encontrar um item e o que mostrar quando nada precisa de reposição. Cada decisão se transforma em uma condição que pode ser testada. Assim, o projeto aproxima lógica, dados e validação sem exigir banco de dados, interface gráfica ou conceitos avançados.

2 OBJETIVOS E CRITÉRIOS DE SUCESSO

2.1 Objetivo geral

Desenvolver e documentar um programa introdutório em Python para cadastrar materiais de um laboratório simulado, registrar entradas e saídas e avisar quando a quantidade disponível estiver no limite mínimo ou abaixo dele.

2.2 Objetivos específicos

- Representar os materiais em uma lista e guardar os dados de cada item em um dicionário.
- Permitir o cadastro e a consulta dos materiais por meio de um menu de texto.
- Atualizar a quantidade ao registrar uma entrada ou uma saída válida.
- Impedir o cadastro duplicado e a retirada de quantidade maior que o saldo disponível.
- Apresentar um aviso quando a quantidade for menor ou igual ao limite mínimo.
- Verificar as regras principais por meio de cenários de teste reproduzíveis.

2.3 Critérios de sucesso

O objetivo é considerado atendido nesta versão quando o programa inicia sem erro, permite cadastrar e listar um item, altera o saldo em uma entrada ou saída válida, recusa uma retirada acima do saldo e mostra o item no relatório de reposição quando o valor fica igual ou inferior ao mínimo. Esses critérios verificam o funcionamento essencial, mas não equivalem à avaliação de uso em um laboratório real.

Não fazem parte do critério de sucesso a preservação dos dados após o encerramento, o registro de usuários, a consulta por interface gráfica ou a emissão de relatórios financeiros. Tais recursos aumentariam o escopo e não são necessários para demonstrar a regra central. A delimitação ajuda a manter o trabalho compatível com uma primeira atividade de programação.

3 FUNDAMENTAÇÃO: CONTROLE DE MATERIAIS

Em sentido prático, controlar materiais envolve identificar os itens, registrar quantidades e atualizar os valores quando ocorrem entradas ou saídas. Uma lista atualizada pode apoiar a consulta e chamar atenção para um saldo baixo. Em organizações maiores, a gestão de estoque pode envolver custos, classificação de itens, giro, prazos e métodos de reposição. O material do Sebrae, por exemplo, apresenta técnicas como Curva ABC, PEPS e custo médio e relaciona uma gestão organizada à prevenção de perdas e ao planejamento das compras [1].

O protótipo utiliza somente uma regra introdutória. Cada item tem uma quantidade atual e um valor mínimo. Quando a quantidade atual é menor ou igual ao mínimo, o programa mostra que o item precisa de reposição. Essa condição sinaliza uma situação para análise; não determina a quantidade a comprar, não calcula prazo de reposição e não faz pedido automático. A decisão continua com a pessoa responsável.

3.1 Diferença entre aviso e decisão de compra

Um aviso é uma informação simples gerada a partir dos números cadastrados. A decisão de compra exige outros dados, como consumo, tempo de entrega, orçamento e regras da organização. Como o sistema não guarda histórico de consumo nem cadastro de fornecedores, ele não tem base para recomendar uma compra. Essa limitação é intencional: evita apresentar como inteligente ou completo um programa que apenas compara dois números.

3.2 Aplicação ao cenário didático

No exemplo escolhido, o material “Cabo HDMI” começa com quatro unidades e mínimo igual a uma. Uma saída de três unidades reduz o saldo para uma. Como a regra considera o mínimo inclusive, o sistema deve exibir o item entre os materiais para repor. Se a quantidade fosse zero, o aviso continuaria aparecendo; se fosse dois, não apareceria. O exemplo mostra a regra sem depender de dados de uma empresa ou de uma pesquisa de campo.

4 FUNDAMENTAÇÃO: CONCEITOS INICIAIS DE PYTHON

Python é usado neste trabalho porque permite escrever um programa textual com estruturas que podem ser estudadas separadamente. A documentação oficial descreve listas como coleções ordenadas às quais elementos podem ser acrescentados e dicionários como pares entre uma chave e um valor [3]. O programa aproveita essas ideias: a lista chamada materiais reúne todos os itens cadastrados; cada item é um dicionário com as chaves nome, quantidade e minimo.

As funções organizam tarefas distintas. buscar_material procura um item; adicionar_material valida e inclui um cadastro; movimentar_material calcula o novo saldo; listar_materiais e mostrar_materiais_para_repor apresentam informações. Essa separação não torna o código complexo: cada função tem um objetivo limitado, e o menu principal apenas escolhe qual delas executar. A documentação oficial de Python apresenta a definição de funções como uma maneira de agrupar instruções reutilizáveis [2].

As decisões condicionais if, elif e else selecionam o caminho adequado. Por exemplo, movimentar_material verifica se o tipo informado é entrada ou saída e, no segundo caso, compara a quantidade solicitada com o saldo. Os laços for percorrem a lista de materiais; o laço while mantém o menu em execução e repete a leitura de números até que o valor seja válido. Essas estruturas de controle são explicadas no tutorial oficial da linguagem [2].

A entrada digitada é recebida como texto e convertida para inteiro. Como uma conversão pode falhar, ler_numero usa try/except para capturar ValueError e pedir uma nova resposta. Exceções representam situações de erro encontradas durante a execução; o tutorial oficial mostra como try/except permite tratar casos esperados sem encerrar imediatamente o programa [4]. O projeto usa essa abordagem apenas para falhas previstas, mantendo mensagens diretas para quem está usando o menu.

O conjunto de conceitos foi escolhido para uma primeira versão. Não há classes, banco de dados, biblioteca externa, interface gráfica, autenticação ou algoritmos de otimização. O foco é perceber como dados, decisões, repetição e validação trabalham juntos para resolver uma regra pequena e verificável.

5 METODOLOGIA E CENÁRIO

O trabalho foi desenvolvido como um protótipo didático, a partir de um cenário simulado de laboratório de informática. A investigação combinou leitura de fontes sobre gestão de estoque, consulta à documentação oficial da linguagem Python e implementação incremental de uma solução em terminal. Não foram realizadas entrevistas, observações em campo, coleta de inventário de uma instituição nem avaliação com usuários. Portanto, os resultados relatados dizem respeito ao comportamento do código em cenários de teste, não ao impacto em uma organização.

5.1 Etapas realizadas

1. Delimitação do problema: registrar uma quantidade e comparar o saldo com um limite mínimo.

2. Definição dos dados: nome, quantidade disponível e quantidade mínima para cada material.

3. Implementação: criação das funções de busca, cadastro, movimentação, listagem e menu.

4. Verificação: execução de cenários válidos e inválidos para conferir as regras principais.

5. Documentação: registro do escopo, do funcionamento, das limitações e de como executar o arquivo.

5.2 Ambiente e ferramentas

O programa foi conferido com Python 3.12.3 em um ambiente de linha de comando. A implementação utiliza somente recursos que acompanham a instalação padrão de Python e não exige pacote externo. O arquivo principal chama-se controle_materiais.py. Para executar, o computador precisa ter Python 3 instalado e acesso ao arquivo baixado do repositório público indicado na capa.

6 REQUISITOS E DELIMITAÇÃO DO ESCOPO

Os requisitos abaixo descrevem o que esta versão deve fazer. Eles foram redigidos de forma observável para que cada item possa ser conferido no menu ou nas funções do programa.

Código	Requisito funcional
RF01	Cadastrar um material com nome, quantidade atual e mínimo.
RF02	Recusar nome vazio e nome já existente, sem distinguir maiúsculas/minúsculas.
RF03	Listar nome, saldo, mínimo e aviso correspondente.
RF04	Registrar entrada positiva e somá-la ao saldo.
RF05	Registrar saída positiva quando houver saldo suficiente.
RF06	Recusar saída maior que o saldo e informar o motivo.
RF07	Mostrar itens com quantidade menor ou igual ao mínimo.
RF08	Repetir o menu até a escolha de encerramento.

Categoria	Regra ou limite
Ambiente	Execução em terminal com Python 3, sem bibliotecas externas.
Persistência	Os dados existem somente durante a execução; não são gravados em arquivo.
Uso	Mensagens em português e menu numerado para facilitar a demonstração.
Fora do escopo	Login, banco de dados, histórico, interface gráfica, compras e relatórios financeiros.

A delimitação evita confundir o protótipo com um sistema completo. Os requisitos atendidos se restringem ao registro temporário e à regra de aviso. Recursos fora do escopo poderão ser discutidos em versões futuras, se forem compatíveis com os conteúdos estudados e aprovados pelo professor.

7 MODELO DE DADOS E LÓGICA DO PROGRAMA

7.1 Estrutura dos dados

Elemento	Tipo usado	Exemplo
materiais	Lista	[item1, item2]
material	Dicionário	{nome, quantidade, minimo}
nome	Texto	Cabo HDMI
quantidade	Inteiro não negativo	4
minimo	Inteiro não negativo	1

A lista permite reunir vários cadastros em uma única coleção. O dicionário agrupa valores que pertencem ao mesmo material. A busca percorre a lista e compara o nome informado com o nome armazenado, removendo espaços nas extremidades e ignorando diferenças entre letras maiúsculas e minúsculas. Isso evita, por exemplo, tratar “Cabo HDMI” e “cabo hdmi” como dois itens diferentes.

7.2 Regra de movimentação

Uma entrada soma a quantidade informada ao saldo existente. Uma saída só é permitida se a quantidade pedida for menor ou igual ao saldo. Depois de uma operação válida, o novo saldo é devolvido pela função para ser mostrado no menu. Quando a saída é maior que o saldo, a função levanta um erro de valor antes de alterar a quantidade; assim, a operação inválida não reduz o saldo.

7.3 Algoritmo em linguagem simples

1. Iniciar uma lista vazia de materiais.
2. Mostrar as opções do menu e ler a escolha.
3. Se a escolha for cadastrar, ler dados, validar e incluir o material.
4. Se for listar ou consultar reposição, percorrer os itens e apresentar os resultados.
5. Se for movimentar, localizar o item, validar a quantidade e atualizar o saldo.
6. Repetir o menu até que a pessoa escolha sair.

8 ORGANIZAÇÃO E IMPLEMENTAÇÃO DO CÓDIGO

O programa foi mantido em um único arquivo para facilitar a leitura e a execução. As funções trabalham sobre a lista recebida como argumento, o que deixa mais visível quais partes procuram, validam ou alteram os dados. A tabela resume a responsabilidade de cada função.

Função	Responsabilidade no protótipo
buscar_material	Procura o item pelo nome e retorna o dicionário ou None.
adicionar_material	Valida o nome e as quantidades; inclui o novo item.
movimentar_material	Soma entrada ou subtrai saída válida.
ler_numero	Repete a pergunta até receber inteiro dentro do mínimo aceito.
listar_materiais	Mostra a lista e a indicação de reposição.
cadastrar_pelo_menu	Lê as respostas e chama a função de cadastro.
fazer_movimentacao	Localiza o item e registra entrada ou saída.
mostrar_materiais_para_repor	Apresenta itens no mínimo ou abaixo dele.
main	Mantém o menu ativo e encaminha cada escolha.

8.1 Validações presentes

O cadastro remove espaços externos do nome, não aceita texto vazio e verifica duplicidade. Quantidade inicial e mínimo não podem ser negativos. Para as movimentações, o número precisa ser maior que zero. A saída também verifica se há saldo suficiente. Na leitura de números, uma entrada que não pode ser convertida em inteiro provoca ValueError; a função captura esse caso, mostra uma mensagem e tenta novamente [4].

8.2 Por que o código não foi ampliado

A lista em memória, o menu de texto e as funções curtas são escolhas adequadas ao objetivo de demonstrar fundamentos. Aumentar a quantidade de telas ou adicionar cadastro de usuários não melhoraria a explicação da regra principal e poderia introduzir temas ainda não estudados. A documentação é mais extensa para contextualizar e explicar o protótipo; o código permanece introdutório.

9 GUIA DE EXECUÇÃO E OPERAÇÃO

9.1 Como iniciar

É necessário instalar Python 3 ou utilizar um computador que já o tenha. Depois de baixar controle_materiais.py, abre-se o terminal na pasta do arquivo e executa-se python3 controle_materiais.py. Em algumas instalações Windows, o comando pode ser py controle_materiais.py. Nenhuma biblioteca externa precisa ser instalada.

9.2 Sequência de demonstração

1. Escolher “1 - Cadastrar material”. Informar Cabo HDMI, quantidade 4 e mínimo 1.
2. Escolher “4 - Registrar saída/uso”. Informar Cabo HDMI e quantidade 3.
3. Escolher “5 - Ver materiais para repor”. O saldo 1 deve aparecer no aviso.
4. Escolher “2 - Listar materiais” para conferir o saldo e o mínimo.
5. Escolher “0 - Sair” para encerrar o programa.

9.3 Respostas a situações comuns

Se a opção do menu não existir, o programa mostra uma mensagem e retorna ao menu. Se o nome buscado não estiver cadastrado, a movimentação não é realizada. Se uma quantidade informada não for inteira, a leitura pede outro valor; se for menor que o mínimo permitido, também pede novamente. Uma tentativa de retirar mais do que o saldo apresenta uma explicação e mantém a quantidade anterior.

A lista permanece disponível apenas enquanto o programa está aberto. Antes de encerrar uma demonstração, é possível consultar os materiais cadastrados; depois de iniciar novamente, os cadastros anteriores não existirão. Esse comportamento deve ser explicado, pois não se trata de falha inesperada, mas de uma limitação assumida na primeira versão.

O arquivo .py é a versão executável. A cópia do código no Apêndice A serve para leitura dentro do relatório; o repositório público indicado na capa oferece o arquivo fonte original e as instruções de execução.

10 TESTES E RESULTADOS

Foram verificados cenários de cadastro, busca, movimento e aviso de reposição com Python 3.12.3. Os testes abaixo são verificações funcionais do protótipo; não representam uma avaliação formal com usuários ou um ensaio em ambiente real.

T01 — Cadastro válido: Cabo HDMI com saldo 4 e mínimo 1 foi incluído.

T02 — Duplicidade: o mesmo nome em letras minúsculas foi recusado.

T03 — Entrada: adicionar 2 unidades ao saldo 4 produziu saldo 6.

T04 — Saída válida: retirar 5 unidades do saldo 6 produziu saldo 1.

T05 — Limite mínimo: com saldo 1 e mínimo 1, o item apareceu para reposição.

T06 — Saída acima do saldo: retirar 2 do saldo 1 foi recusado; o saldo permaneceu 1.

T07 — Entrada inválida: texto foi recusado e a função aceitou um inteiro válido na nova tentativa.

T08 — Menu integrado: cadastro, saída, aviso e encerramento ocorreram sem erro.

Os resultados confirmam as regras verificadas neste conjunto de casos. Eles não provam ausência de todo tipo de defeito. Não foram avaliados desempenho com listas extensas, acessibilidade, segurança de dados, uso simultâneo, recuperação após falha ou compatibilidade com versões antigas de Python. Para uma próxima etapa, esses aspectos poderiam ser transformados em critérios adicionais.

11 LIMITAÇÕES, SEGURANÇA E MELHORIAS

11.1 Limitações conhecidas

A principal limitação é a ausência de armazenamento permanente. Os materiais são guardados em uma lista criada quando o programa inicia e desaparecem quando ele termina. O código também não mantém histórico de movimentações, não registra quem retirou cada item, não protege os dados com senha e não sincroniza informações entre computadores. Como o nome é a forma de localizar o item, não existe código numérico ou identificador separado.

11.2 Uso responsável

O protótipo não deve ser usado como único registro de materiais de uma instituição. Informações importantes precisam de controle autorizado e cópia segura. Como não há autenticação nem trilha de auditoria, qualquer pessoa com acesso ao terminal poderia alterar os dados enquanto o programa está aberto. O cenário simulado não contém informações pessoais; uma adaptação real precisaria avaliar quais dados seriam coletados e quem poderia consultá-los.

11.3 Evoluções possíveis

Uma evolução inicial poderia gravar a lista em um arquivo local, desde que esse conteúdo já faça parte dos estudos e o orientador aprove a mudança. Em seguida, poderiam ser avaliados identificadores simples, histórico de entradas e saídas e validação de quantidades por um responsável. Interface gráfica, vários usuários, banco de dados, cópias de segurança e relatórios automáticos são propostas futuras, não recursos presentes nesta entrega.

A ordem das melhorias deve priorizar a necessidade real e a capacidade de manter o programa. Antes de aumentar o escopo, seria adequado confirmar regras com o orientador e, se houver autorização, conversar com quem conhece o processo. A consulta bibliográfica sobre controle de estoque pode ajudar a formular perguntas, mas não substitui a validação local [1] [5].

12 CONSIDERAÇÕES FINAIS

O objetivo de desenvolver e documentar um programa introdutório para controle de materiais foi alcançado dentro do escopo definido. A aplicação cadastra e lista itens, registra entradas e saídas, recusa valores que produziriam saldo negativo e indica quando a quantidade alcança o mínimo. A verificação funcional confirmou os cenários descritos na seção de testes e a execução integrada do menu.

A pergunta do projeto foi respondida de forma delimitada: um programa simples pode apoiar o registro temporário de materiais e destacar um saldo baixo por meio de uma comparação entre quantidade atual e mínimo. Essa resposta não significa que o sistema substitui uma ferramenta profissional. Sem persistência, usuários, histórico e validação de regras de uma instituição, o programa continua sendo um exercício didático.

A proposta permitiu aplicar conceitos iniciais de Python em um problema específico. Listas e dicionários organizam os dados; funções separam tarefas; condicionais selecionam ações; laços mantêm o menu e percorrem cadastros; exceções ajudam a tratar entradas inválidas. O código foi mantido enxuto para que cada parte possa ser explicada e revisada por quem está no início do curso [2] [3] [4].

Como continuidade, recomenda-se apresentar o cenário ao professor orientador e confirmar que a documentação atende à etapa solicitada. Se houver autorização para desenvolver uma versão aplicada, o primeiro passo é levantar as necessidades reais e decidir se a preservação de dados é necessária. Assim, novas funções seriam acrescentadas com justificativa e sem transformar uma atividade introdutória em um sistema além do nível estudado.

REFERÊNCIAS

[1] SERVIÇO BRASILEIRO DE APOIO ÀS MICRO E PEQUENAS EMPRESAS (SEBRAE/RN). Gestão de estoque. E-book Finanças. Disponível em: https://digital.rn.sebrae.com.br/conteudos/ebook/gestao-de-estoque. Acesso em: 4 nov. 2026.

[2] PYTHON SOFTWARE FOUNDATION. The Python Tutorial: More Control Flow Tools. Disponível em: https://docs.python.org/3/tutorial/controlflow.html. Acesso em: 4 nov. 2026.

[3] PYTHON SOFTWARE FOUNDATION. The Python Tutorial: Data Structures. Disponível em: https://docs.python.org/3/tutorial/datastructures.html. Acesso em: 4 nov. 2026.

[4] PYTHON SOFTWARE FOUNDATION. The Python Tutorial: Errors and Exceptions. Disponível em: https://docs.python.org/3/tutorial/errors.html. Acesso em: 4 nov. 2026.

[5] UNIVERSIDADE SANTO AMARO. Manual para elaboração de Projeto Integrador: Laboratório de Programação. São Paulo: UNISA, 2024. Material fornecido para orientação da disciplina.

APÊNDICE A — CÓDIGO-FONTE

O código a seguir é uma cópia para consulta do arquivo controle_materiais.py disponível no repositório público. Para executar o programa, deve-se baixar o arquivo .py e usar o comando indicado na seção 9. A reprodução no PDF facilita a entrega em um único arquivo, mas não substitui o código executável.

```
"""Controle simples de materiais para um laboratório de informática.
Executar com: python3 controle_materiais.py
Os dados existem apenas enquanto o programa está aberto.
"""
def buscar_material(materiais, nome):
    """Procura um material pelo nome, sem diferenciar maiúsculas."""
    for material in materiais:
        if material["nome"].lower() == nome.strip().lower():
            return material
    return None
def adicionar_material(materiais, nome, quantidade, minimo):
    """Valida e adiciona um material à lista."""
    nome = nome.strip()
    if nome == "":
        raise ValueError("O nome não pode ficar vazio.")
    if buscar_material(materiais, nome) is not None:
        raise ValueError("Já existe um material com esse nome.")
    if quantidade < 0 or minimo < 0:
        raise ValueError("As quantidades não podem ser negativas.")
    material = {"nome": nome, "quantidade": quantidade, "minimo": minimo}
    materiais.append(material)
    return material
def movimentar_material(material, tipo, quantidade):
    """Registra uma entrada ou saída e atualiza a quantidade."""
    if quantidade <= 0:
        raise ValueError("Informe uma quantidade maior que zero.")
    if tipo == "entrada":
        material["quantidade"] += quantidade
    elif tipo == "saida":
        if quantidade > material["quantidade"]:
            raise ValueError("Não há material suficiente para essa saída.")
        material["quantidade"] -= quantidade
    else:
        raise ValueError("Tipo de movimentação inválido.")
    return material["quantidade"]
def ler_numero(pergunta, minimo=0):
    """Lê um número inteiro e repete a pergunta enquanto for inválido."""
    while True:
        try:
            numero = int(input(pergunta))
            if numero < minimo:
                print("Digite um número maior ou igual a", minimo)
            else:
                return numero
        except ValueError:
            print("Digite um número inteiro.")
def listar_materiais(materiais):
    if len(materiais) == 0:
        print("Ainda não há materiais cadastrados.")
        return
    print("\nMATERIAL | QUANTIDADE | MÍNIMO")
    for material in materiais:
        aviso = " - precisa repor" if material["quantidade"] <= material["minimo"] else ""
        print(material["nome"], "|", material["quantidade"], "|", material["minimo"], aviso)
def cadastrar_pelo_menu(materiais):
    nome = input("Nome do material: ")
    quantidade = ler_numero("Quantidade atual: ")
    minimo = ler_numero("Avisar quando chegar a (mínimo): ")
    adicionar_material(materiais, nome, quantidade, minimo)
    print("Material cadastrado.")
def fazer_movimentacao(materiais, tipo):
    nome = input("Nome do material: ")
```

```
    material = buscar_material(materiais, nome)
    if material is None:
        print("Material não encontrado. Confira o nome na lista.")
        return
    quantidade = ler_numero("Quantidade: ", 1)
    try:
        saldo = movimentar_material(material, tipo, quantidade)
        print("Movimentação registrada. Quantidade atual:", saldo)
    except ValueError as erro:
        print(erro)
def mostrar_materiais_para_repor(materiais):
    encontrados = False
    print("\nMateriais no mínimo ou abaixo dele:")
    for material in materiais:
        if material["quantidade"] <= material["minimo"]:
            print("-", material["nome"], "(quantidade:", material["quantidade"], ")")
            encontrados = True
    if not encontrados:
        print("Nenhum material precisa de reposição agora.")
def main():
    materiais = []
    while True:
        print("\n=== CONTROLE DE MATERIAIS DO LABORATÓRIO ===")
        print("1 - Cadastrar material")
        print("2 - Listar materiais")
        print("3 - Registrar entrada")
        print("4 - Registrar saída/uso")
        print("5 - Ver materiais para repor")
        print("0 - Sair")
        opcao = input("Escolha uma opção: ").strip()
        try:
            if opcao == "1":
                cadastrar_pelo_menu(materiais)
            elif opcao == "2":
                listar_materiais(materiais)
            elif opcao == "3":
                fazer_movimentacao(materiais, "entrada")
            elif opcao == "4":
                fazer_movimentacao(materiais, "saida")
            elif opcao == "5":
                mostrar_materiais_para_repor(materiais)
            elif opcao == "0":
                print("Programa encerrado.")
                break
            else:
                print("Opção inválida. Escolha uma opção do menu.")
        except ValueError as erro:
            print(erro)
if __name__ == "__main__":
    main()
```